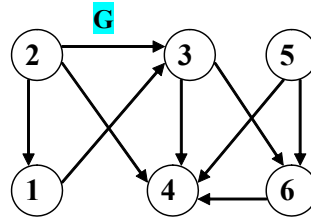


TD1&2**Exercice 1-Facultatif**

Soit G le graphe orienté ci-contre :

- Donner pour chaque sommet du graphe :
 - Les successeurs et les prédécesseurs
 - Les demi-degrés intérieurs et extérieurs
 - Les voisins de chaque sommet
- Représenter le graphe par :
 - Matrice d'incidence
 - Matrice d'adjacence
- Représentation sous forme condensée de la matrice d'adjacence :
 - File des successeurs
 - Listes d'adjacence
 - Tableau de listes successeurs et prédécesseurs
 - Liste principale (représentation par pointeur de la matrice d'adjacence)
 - Matrice creuse
- Représentation sous forme condensée de la matrice d'incidence :
 - Représentation colonne par colonne
 - Représentation ligne par ligne
 - Listes d'incidences
- Décrire en langage C les structures de données nécessaires à la représentation des graphes par les représentations ci-dessus.

**EXERCICE 2**

Pour chacune des représentations ci-dessous, retrouver le graphe correspondant :

0	0	0	0	0	0	1
0	0	0	0	1	0	0
0	0	0	0	0	1	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	0	1	0	0	0	0
1	0	0	0	1	0	0

Matrice d'Adjacence

APS

1	1	2	3	4	5	6	6
---	---	---	---	---	---	---	---

FS

5	5	1	7	4
---	---	---	---	---

File des Successeurs

APS

1	5	9	12
---	---	---	----

FS

1	2	3	6	1	2	3	4	4	5	6
---	---	---	---	---	---	---	---	---	---	---

AS

2	2	2	3	1	1	1	3	2	3	1
---	---	---	---	---	---	---	---	---	---	---

Matrice d'incidence sous forme condensée
ligne par ligne

Exercice 3

On considère un **graphe orienté** $G=(S, A)$ qui peut être représenté selon l'une des structures suivantes :

- Une matrice d'adjacence.
- Une liste d'adjacence
- Une file des successeurs.

Pour chaque type de représentation, écrire les algorithmes suivants :

- Détermination des successeurs** : Écrire un algorithme qui identifie tous les successeurs d'un sommet s donné dans un graphe G donné.
- Détermination des prédécesseurs** : Écrire un algorithme qui identifie tous les prédécesseurs d'un sommet s donné dans un graphe G donné.
- Détection de boucles** : Écrire un algorithme qui vérifie si un graphe G donné contient une boucle (un arc reliant un sommet à lui-même).
- Sommet ayant le plus de successeurs** : Écrire un algorithme qui détermine le sommet du graphe G non vide, non vide, ayant le plus grand nombre de successeurs.

Exercice 4

- Écrire un algorithme qui construit la liste d'adjacence d'un graphe donné à partir de sa matrice d'adjacence.
- Écrire un algorithme qui construit la file des successeurs d'un graphe donné à partir de sa matrice d'adjacence.
- Écrire un algorithme qui construit le tableau de listes des successeurs et prédécesseurs d'un graphe donné à partir de sa matrice d'adjacence.
- Écrire un algorithme qui construit la liste d'adjacence d'un graphe donné à partir de sa file des successeurs.
- Écrire un algorithme qui construit la file des successeurs d'un graphe donné à partir de la liste d'adjacence.
- Écrire en langage algorithmique (ou en langage C), le passage d'un **graphe orienté** $G=(S, A)$ représenté par sa une matrice d'adjacence à une autre représentation par une matrice d'incidence.
- Écrire en langage algorithmique (ou en langage C), le passage d'un **graphe orienté** $G=(S, A)$ représenté par sa une matrice d'incidence à une autre représentation par une matrice d'adjacence.
- Écrire en langage algorithmique (ou en langage C), le passage d'un **graphe non orienté** $G=(S, A)$ représenté par sa une matrice d'incidence à une autre représentation par une matrice d'adjacence.

Exercice 5

Soit $G=(S, A)$ un graphe.

- Montrer par récurrence que la somme des degrés des sommets de G est égale à deux fois le nombre d'arêtes si le graphe est non orienté et deux fois le nombre d'arcs si le graphe est orienté.
- En déduire qu'un graphe (orienté ou non orienté) a un nombre pair de sommets de degré impair.
- Que peut-on dire de plus dans le cas d'un graphe orienté ?

TP

- 1) Ecrire en C sous forme de fonctions puis procédures, les algorithmes suivants :
 - ✓ Allocation mémoire d'un tableau composé de n éléments qui sont de type de TElement.
 - ✓ Libération mémoire d'un tableau composé de n éléments qui sont de type de TElement.
 - ✓ Allocation mémoire d'une matrice rectangulaire composé de mxn éléments qui sont de type de TElement.
 - ✓ Libération mémoire d'une matrice rectangulaire composé de mxn éléments qui sont de type de TElement.
 - ✓ Allocation de la structure d'une matrice d'adjacence (resp matrice d'incidence).
 - ✓ Libération de la structure d'une matrice d'adjacence (resp matrice d'incidence).
- 2) Implémenter les différentes techniques de représentations du graphe.
 - ✓ Ecrire une fonction **Chargement** recevant en paramètre d'entrée une chaîne de caractères représentant le nom d'un fichier de texte contenant les données d'un graphe, et ayant comme paramètre résultat un GRAPHE constitué à partir de ces données.
 - ✓ Ecrire une fonction **Enregistrement** (opération inverse de chargement) qui sauvegarde dans un fichier texte la structure du graphe.

Annexe : Manipulations de base des fichiers en C

Les fichiers sont représentés dans les programmes C par le type FILE, défini dans <stdio.h>

Un fichier se manipule en trois étapes classiques:

1. Ouverture du fichier. Cela établit le lien entre la variable du programme (de type FILE* en général) et le fichier physique.
2. Lectures, écritures et déplacements. En fonction du type de fichier (texte ou binaire), on aura recours à deux fonctions différentes pour les lectures/écritures.
3. Fermeture du fichier. Cela rompt le lien établi à l'ouverture, et garantit que les modifications apportées au fichier pendant l'étape 2 seront bien écrites sur le disque.

Ouverture d'un fichier

L'ouverture se fait à l'aide de la fonction fopen:

```
FILE* fopen(char* nomFichier, char* mode);
```

le mode précise les opérations qu'on souhaite faire sur le contenu du fichier, comme résumé dans le tableau suivant:

mode	signification
r	lecture seule
w	écriture seule. N.B. : RAZ du fichier!
a	écriture seule, après les données existant dans le fichier
r+	lecture et écriture, positionnement en début de fichier
w+	lecture et écriture, mais RAZ du fichier au départ!
a+	lecture et écriture, positionnement en fin de fichier

Lecture / écriture de fichier "texte"

On utilise les fonctions **fscanf** et **fprintf**, similaires à scanf et printf dans leur comportement:

- `int fscanf(FILE* fichier, char* format, ...);`
Cette fonction renvoie le nombre d'éléments effectivement lus, ou EOF si aucune lecture n'était possible.
- `int fprintf(FILE* fichier, char * format, ...);`
Cette fonction renvoie le nombre de caractères effectivement écrits (sans le '\0' final)

Lecture / écriture de fichier binaire

On utilise les fonctions **fread** et **fwrite** suivantes:

- `size_t fread( void *buffer, size_t tailleUnitaire, size_t nbElts, FILE * fichier);`
Cette fonction lit *nbElts* objets de taille *tailleUnitaire* dans le fichier *fichier*, et stocke leur valeur dans la zone pointée par *buffer*. Elle renvoie le nombre d'objets effectivement lus ($\leq nbElts$)
- `size_t fwrite( const void *buffer, size_t tailleUnitaire, size_t nbElts, FILE *fichier);`
Cette fonction écrit dans *fichier* *nbElts* objets de taille *tailleUnitaire*. Ces objets ont été placés à l'adresse *buffer* au préalable. Elle renvoie le nombre d'objets effectivement écrits ($\leq nbElts$)

Déplacements dans un fichier

On utilise les fonctions **fseek** et **ftell** pour, respectivement, se déplacer et savoir où on est dans un fichier:

- `int fseek( FILE * fichier, long offset, int base);`
Cette fonction déplace la tête de lecture / écriture du fichier de *offset* octets à partir de *base*. *base* peut prendre trois valeurs (`SEEK_SET`, début du fichier - `SEEK_CUR`, position courante - `SEEK_END`, fin du fichier). Elle renvoie 0 en cas de succès et -1 en cas d'échec.
- `long ftell( FILE * fichier);`
Cette fonction renvoie la position courante de la tête de lecture/écriture dans le fichier.

Fermeture de fichier

On utilise la fonction **fclose** :

- `int fclose( FILE * fichier);`
Cette fonction renvoie 0 en cas de succès, EOF en cas d'échec.